

Übung 6: Alternative Quellen vergleichen und Abweichungen analysieren

Voraussetzungen

- **RStudio-Projekt:** `projekt/vgr-r-schulung.Rproj` öffnen. Datenzugriff relativ als `data/...`
- **Pakete:** `dplyr`, `readr`, `waldo`, `diffdf` (installiert).
- **Daten:** `data/lieferung_2024q4.csv` (Vintage 1), `data/lieferung_2025q1.csv` (Vintage 2).
- **Startdatei:** `R/lab6_abweichungsanalyse.R` (mit `# TODO-Lücken`).

Zur Selbstkontrolle: Beide Stände haben 280 gemeinsame Schlüssel. Das neue Quartal 2025Q1 liefert 10 zusätzliche Schlüssel nur in Vintage 2. Beim Schwellenwert von 0,5 Prozent sind 36 gemeinsame Zeilen auffällig.

Datensätze

Beide Dateien enthalten die Spalten `jahr`, `quartal`, `bereich_code`, `bereich_name`, `wert` (Mio. EUR, jeweilige Preise).

- **Vintage 1** (`lieferung_2024q4.csv`): Datenstand 2024Q4, 2018Q1 bis 2024Q4.
- **Vintage 2** (`lieferung_2025q1.csv`): Datenstand 2025Q1. Die Jahre 2023/2024 wurden leicht revidiert, und das Quartal 2025Q1 ist neu hinzugekommen.

Der fachliche Schlüssel ist die Kombination `jahr / quartal / bereich_code`. `bereich_name` ist nur eine Beschriftung und gehört nicht in den Schlüssel.

Schritt 1: Einleseprozess als Funktion parametrisieren

Statt zweimal `read_csv()` zu kopieren, kapseln Sie das Einlesen in eine Funktion `lade(pfad)`. So lesen Sie beide Quellen mit identischer Logik ein. Wichtig: Spaltentypen explizit festlegen, damit die Schlüsselspalten in beiden Ständen gleich typisiert sind. Bei inkompatiblen Typen bricht `dplyr` mit einer Fehlermeldung ab; ein einheitlicher Import verhindert diesen Fehler.

```
lade <- function(pfad) {  
  # TODO: CSV mit expliziten Spaltentypen einlesen.  
  NULL  
}  
  
vintage1 <- lade("data/lieferung_2024q4.csv")  
vintage2 <- lade("data/lieferung_2025q1.csv")
```

Lesen Sie beide Quellen mit derselben Funktion ein. Wenn `quartal` einmal als Text und einmal als Zahl eingelesen wird, meldet `dplyr` inkompatible Schlüsseltypen und bricht den Join ab.

Schritt 2: Beide Stände zusammenführen (`inner_join`)

`inner_join` behält nur Schlüssel, die in **beiden** Ständen vorkommen. Mit `suffix = c("_v1", "_v2")` werden die gleichnamigen Wertspalten zu `wert_v1` und `wert_v2`.

```
schluessel <- c("jahr", "quartal", "bereich_code")  
gemeinsam <- NULL # TODO: inner_join mit passenden Suffixen
```

Schritt 3: Abweichungen berechnen und markieren

Berechnen Sie die absolute Abweichung ($\text{wert_v2} - \text{wert_v1}$) und die relative Abweichung (bezogen auf den alten Stand wert_v1). Markieren Sie eine Zeile als auffällig, wenn der Betrag der relativen Abweichung den Schwellenwert (hier 0,5 %, also 0.005) überschreitet.

```
schwelle <- 0.005

abweichungen <- gemeinsam |>
  mutate(
    # TODO: absolute Abweichung,
    # TODO: relative Abweichung,
    # TODO: Kennzeichen für auffällige Werte
  ) |>
  select(
    jahr, quartal, bereich_code, bereich_name = bereich_name_v1,
    wert_v1, wert_v2, abw_abs, abw_rel, auffaellig
  )
```

Schritt 4: anti_join in beide Richtungen

`anti_join(A, B)` liefert die Zeilen aus A, deren Schlüssel **nicht** in B vorkommen. In beide Richtungen ausgeführt zeigt das, welche Perioden nur in einem Stand existieren.

```
nur_in_v2 <- NULL # TODO: anti_join von Vintage 2 gegen Vintage 1
nur_in_v1 <- NULL # TODO: anti_join von Vintage 1 gegen Vintage 2
```

Erwartung: 2025Q1 ist nur in Vintage 2 vorhanden, in Vintage 1 fehlt es.

Schritt 5: Objekt- und Tabellenvergleich

Zwei komplementäre Werkzeuge:

- `waldo::compare()` vergleicht zwei R-Objekte strukturell und werteweise und zeigt die Unterschiede zeilen-genau. Gut für einen kleinen Ausschnitt.
- `diffdf::diffdf()` vergleicht zwei `data.frames` über Schlüsselspalten und zählt die abweichenden Werte je Variable. Gut für den vollen Datensatz.

```
ausschnitt_v1 <- NULL # TODO: 2024Q4 aus Vintage 1 auswählen
ausschnitt_v2 <- NULL # TODO: 2024Q4 aus Vintage 2 auswählen

# TODO: Ausschnitte mit waldo::compare() vergleichen.

v1_keys <- NULL # TODO: Schlüssel und wert_v1 auswählen
v2_keys <- NULL # TODO: Schlüssel und wert_v2 auswählen

# TODO: Gemeinsame Schlüssel mit diffdf::diffdf() vergleichen.
```

`diffdf::diffdf()` gibt eine Warnung aus, wenn nicht alle Werte gleich sind. Das ist hier erwünscht und kein Fehler.

Schritt 6: Report exportieren

Schreiben Sie die auffälligen Zeilen, sortiert nach Betrag der relativen Abweichung (größte Abweichung zuerst), nach `output/abweichungen.csv`. Den Ordner legen Sie vorher robust an.

```
dir.create("output", showWarnings = FALSE)

report <- NULL # TODO: auffällige Zeilen filtern und sortieren

# TODO: Report mit write_csv() exportieren.
```

Ausführung

Vervollständigen Sie `R/lab6_abweichungsanalyse.R` und führen Sie die Datei aus dem Projektordner mit `Rscript R/lab6_abweichungsanalyse.R` aus.

Zusatzaufgaben (optional)

- Senken Sie die Schwelle auf 0,2 % (0.002). Wie ändert sich die Zahl der auffälligen Zeilen?
- Gruppieren Sie die auffälligen Zeilen nach `bereich_code` und zählen Sie, welcher Bereich am häufigsten revidiert wurde.

Hinweise

- Den Schlüssel `schluessel <- c("jahr", "quartal", "bereich_code")` einmal definieren und an `inner_join`, `anti_join` und `diffdf` durchreichen.
- `bereich_name` doppelt sich nach dem Join (`bereich_name_v1 / bereich_name_v2`). Im select einen Namen behalten.
- Relative Abweichung immer auf den alten Stand (`wert_v1`) beziehen, damit die Prozentwerte eine konsistente Basis haben.